VAKRATRON SYSTEMS

WHITEPAPER · APIS AND MICROSERVICES

APIs and Microservices Without the Distributed Monolith

Designing APIs, choosing between a modular monolith and microservices, and running integrations safely

Written for
CIOs, enterprise and solution architects, engineering managers, integration and security teams

Version 1.0, October 2026
vakratronsys.com

Contents

01 Executive summary

02 Why integration gets painful

03 Core concepts: boundaries, contracts and versions

04 Monolith, modular monolith or microservices

05 A decision framework

06 Reference architecture

07 Sizing and cost

08 Security and governance

09 Operating it: day 2

10 A practical roadmap

11 Ten common mistakes

12 API and integration readiness checklist (36 points)

13 Glossary

14 About Vakratron Systems

What you will take away

- How to choose between a monolith, a modular monolith and microservices
- API design and versioning rules that keep callers working
- When to call an API and when to publish an event
- Timeouts, retries, circuit breakers and idempotency, explained simply
- Availability and event sizing formulas with a worked example
- API security based on the OWASP API Top 10, and opening APIs to partners
- A 36-point readiness checklist for integration and API programmes

01

Executive summary

Most integration problems are not about technology. They come from point-to-point connections nobody documented, services split before anyone knew where the boundaries were, and one slow system dragging others down with it. Microservices promised independent teams and faster releases. Many organisations got the opposite: a distributed monolith, where every change still touches several services, but now each call can fail over the network.

This paper is for the people who shape how systems talk to each other: CIOs, enterprise and solution architects, engineering managers, and the integration and security teams who keep it running. It sets out how to choose the right structure, design APIs that last, and make integrations safe to operate.

Five things to take away

1. **Start with a modular monolith.** Split a service out only when teams block each other, or one part needs very different scale or release pace. Microservices solve an organisational problem, not a technical one.
2. **Design the contract first.** Write the API in OpenAPI before the code, and never break existing callers. Add fields freely; remove them only in a new version, with notice.
3. **Use events for news, calls for questions.** When several systems react to the same thing, publish an event. Long chains of synchronous calls multiply failure.
4. **Make failure small.** Every call has a timeout, retries are limited and idempotent, and circuit breakers stop one slow service from taking down the rest.
5. **Know every API you expose.** Most API breaches are missing checks on APIs nobody remembered. Keep a catalogue, put a gateway in front, and check object ownership in every service.

02

Why integration gets painful

Integration estates grow one project at a time. Each connection made sense when it was built; together they form a web that nobody can draw from memory. When we review integration estates, the technology varies but the problems repeat:

What we find	What it leads to
Spaghetti connections	Every system talks directly to every other. Change one, and you find out what breaks in production.
Microservices too early	A small team split one application into twenty services and got all of the complexity with none of the benefit.
Shared databases	Services read and write each other's tables. No service can change its schema without a coordinated release.
Long synchronous chains	One user request passes through six services. Any one of them being slow makes the whole request slow.
No timeouts	A service waits forever on a slow dependency, threads pile up, and the failure spreads to everything that calls it.
Unknown APIs	Old versions and test endpoints stay open on the internet, with no owner and no monitoring.
Breaking changes	A field is renamed, partners' integrations fail overnight, and nobody knew who was calling.

> **The distributed monolith:** services that must be deployed together, share data, and call each other in long chains. It has the coupling of a monolith and the failure modes of a distributed system. Avoiding it is the main aim of this paper.

03

Core concepts: boundaries, contracts and versions

Boundaries come from the business

A good service boundary follows a business capability with its own data and its own language: orders, payments, inventory, customers. Domain-driven design calls this a bounded context. Inside a boundary, things change together and can share a database. Across a boundary, they talk only through a published contract. If two parts of a system always change together, they belong in the same boundary, however large it looks.

Boundaries matter whether you run one deployable unit or fifty. A modular monolith uses the same boundaries as microservices, enforced as modules with separate schemas inside one application. That keeps the option to split later without paying for it now.

Contract-first API design

A good API is one a developer can use correctly from its documentation alone, and one you can change without breaking the people who depend on it.

Rule	Why it matters
Write the contract first, in OpenAPI (AsyncAPI for events)	Callers and providers agree before code exists; documentation, mocks and tests come from the same file.
Model resources, not database tables	The API can stay stable while the storage behind it changes.
One error format everywhere	A clear code, message and correlation ID; callers handle errors the same way for every API.
Idempotency keys on payments and orders	A retried request never charges or ships twice.
Pagination and filters on every list	No endpoint returns everything at once, so performance stays predictable.
Validate every request against the contract	Bad input is rejected at the edge, not deep inside a service.

Versioning without breaking callers

Adding an optional field or a new endpoint is not a breaking change. Removing or renaming a field, changing a type, or tightening validation is. Avoid breaking changes; when one is unavoidable, publish a new major version (for example `/v2/`), run both for an agreed period, usually six to twelve months for partners, tell callers early, and use gateway logs to see exactly who still calls the old version before you retire it.

REST, GraphQL or gRPC

- **REST** for most public and partner APIs: widely understood, easy to cache and secure.
- **GraphQL** when many front-ends need different shapes of the same data, behind a well-governed schema.
- **gRPC** for fast internal service-to-service calls with strict contracts.

04

Monolith, modular monolith or microservices

Microservices solve a specific problem: many teams needing to change and release parts of a system independently. If you do not have that problem, they mostly add cost. The honest comparison:

	Traditional monolith	Modular monolith	Microservices
Best for	Small, stable applications	One to a few teams; a product still taking shape	Many teams; parts with very different scale or release needs
Deployment	One unit	One unit, built from clear modules	Many units; needs a platform and automation
Data	One shared schema	One database, a schema per module	A database per service; harder consistency
Failure	In-process calls rarely fail	In-process calls rarely fail	Every network call can fail and must be handled
Team independence	Low	Moderate	High, if boundaries are right
Running cost	Lowest	Low	Higher: platform, monitoring, gateway, broker, people
Main risk	Becomes a tangle	Module rules not enforced	The distributed monolith

Signs you should split a service out

- Teams wait on each other to release, every week.
- One part needs to scale very differently from the rest, such as search or tracking.
- One part changes daily while the rest is stable.
- A clear business boundary exists, with its own data and its own owner.

Signs you split too early

- Every feature needs changes in several services at once.
- Services share a database, or must be deployed together.
- You cannot test anything without starting everything.

05

A decision framework

Three decisions shape most integration architectures. Make them per domain, not once for the whole organisation.

1. **How many deployable units?** Count the teams that need to release independently. One team rarely needs more than one or two services. Split along business boundaries, one service per team at most to begin with.
2. **Call or event?** If the caller needs an answer to continue, call an API. If the sender is announcing that something happened, publish an event and let others react.
3. **Change or wrap the legacy system?** Put an API or adapter in front first. Rewrite only what blocks the business, one piece at a time.

Situation	Use	Example
Caller needs an answer to continue	API call	Check stock before confirming an order
Several systems react to the same thing	Event	“Order placed” updates warehouse, billing and notifications
The receiver may be offline for a while	Event or queue	A branch system that syncs every few minutes
Very high volume of updates	Event stream	Tracking pings from thousands of vehicles
Long-running work with steps	Queue plus status API	Document processing or loan approval

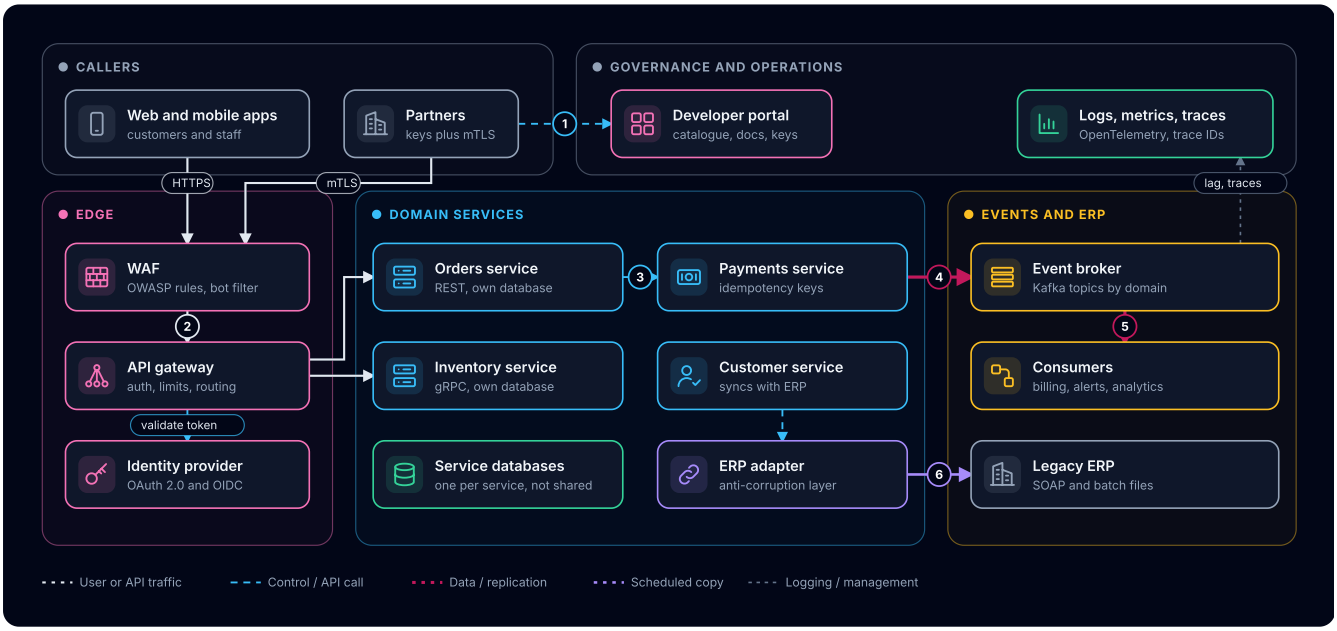
Modernising a legacy system: the strangler pattern

Put a gateway or API in front of the legacy system. Build new capabilities as separate services behind the same front. Move existing features out one at a time, routing traffic to the new version, and retire the old parts as they empty out. Callers never notice, and every step can be rolled back. Big-bang rewrites of core systems fail far more often than they succeed.

06

Reference architecture

The diagram shows a typical shape once a few domains have their own services. Callers only ever reach the gateway. Services call each other sparingly and with timeouts, share news through events, and reach legacy systems only through adapters.



Numbered flows: (1) partners register in the developer portal and receive keys and certificates, (2) all external traffic passes the WAF, then the gateway checks the token with the identity provider, applies rate limits and routes to the right service and version, (3) orders calls payments with a timeout, a limited retry carrying an idempotency key, and a circuit breaker, (4) payments publishes “payment received” through an outbox, (5) billing, alerts and analytics react independently, (6) the legacy ERP is reached only through an adapter that translates its model.

Gateway patterns

Pattern	What it does	Use when
Edge gateway	One entry point for authentication, rate limits, routing and logging	Always, for anything exposed outside
Backend for front-end	A thin API per channel (mobile, web) that shapes data for it	Channels need very different responses
Partner gateway	Separate keys, mTLS, quotas and a portal for partners	You open APIs to third parties
Service mesh	mTLS, retries and traffic control between services, outside the code	Many services, and a platform team to run it

Keep business logic out of the gateway. It should check, limit, route and record. Gateways that start transforming data and orchestrating calls become a new monolith that only one team understands. Common choices are Kong, Apache APISIX, a cloud API gateway, or a Kubernetes Gateway API implementation.

Resilience patterns

Pattern	What it prevents	Rule of thumb
Timeout on every call	Threads waiting forever on a slow dependency	Set from the dependency's p99 latency; always shorter than the caller's own timeout
Retry with backoff and jitter	Failing on brief network blips	Two retries at most, only on safe errors, and only at one layer
Idempotency key	Double charges or duplicate orders when a retry succeeds twice	Client sends a unique key; the service stores and replays the first result
Circuit breaker	Hammering a service that is already down	Open after a run of failures, fail fast, test again after a pause
Bulkhead	One slow dependency using up all threads or connections	Separate pools per dependency
Fallback	A whole page failing because one part did	Show cached or partial data where the business accepts it

Events, done reliably

Kafka suits high-volume streams that several consumers read and may replay; RabbitMQ suits work queues and routing; NATS suits lightweight messaging. Whichever you use, five practices matter more than the product:

- **Outbox:** save the change and the event in the same database transaction, then publish, so the database and the events never disagree.
- **Idempotent consumers:** messages can arrive twice; processing one twice must be harmless.
- **Schemas:** describe events with a schema registry (Avro or JSON Schema) and evolve them without breaking consumers.
- **Dead-letter queues:** messages that keep failing go aside for a person to inspect, instead of blocking the rest.
- **Ordering:** decide where order matters, and partition by the right key, such as order ID.

07

Sizing and cost

Availability of a call chain

When a request depends on several services in sequence, their availabilities multiply:

```
End-to-end availability = A1 × A2 × … × An
```

```
Calls reaching the deepest service = (1 + retries per layer), raised to the power of the number of retrying layers
```

Six services at 99.9 percent each give $0.999^6 \approx 99.4$ percent, or about 52 hours of failed requests a year instead of nine. And if three layers each retry twice, one user click can become 3 × 3 × 3 = 27 calls to the deepest service, exactly when it is struggling. Retry at one layer only.

A worked example: placing an order

An order passes through orders, payments, inventory, customer and notifications, each running at 99.9 percent availability.

	Design A: all synchronous calls	Design B: two calls, the rest as events
Services the user waits on	5	2 (orders and payments)
End-to-end availability	$0.999^5 \approx 99.5\%$	$0.999^2 \approx 99.8\%$
Failed-order time per year	8,760 × 0.005 ≈ 44 hours	8,760 × 0.002 ≈ 18 hours
If notifications is down	Orders fail	Orders succeed; notifications catch up later
Response time	Sum of five calls	Sum of two calls

Sizing an event stream

```
Partitions ≥ peak messages per second ÷ messages per second one consumer can process
```

```
Storage per day = message rate × message size × 86,400 seconds; total = per day × retention days × replicas
```

A logistics company tracks 20,000 vehicles that each send a ping every 30 seconds: about 670 messages a second, or roughly 1,340 at peak. If one consumer handles 200 a second, it needs at least 1,340 ÷ 200 ≈ 7 partitions; plan 12 to 16 to allow growth, because partitions are hard to add later without reordering keys. At 1 KB per message, that is about 115 GB a day; seven days of retention with three replicas needs 115 × 7 × 3 ≈ 2.4 TB.

What microservices add to running cost

Item	Planning range per year
Container platform for 15 to 30 services, production and non-production	₹40 lakh to ₹1 crore in infrastructure
Event broker cluster (three nodes) and schema registry	₹10 to ₹25 lakh
Gateway and developer portal: open source, or commercial licences	₹0 to ₹60 lakh
Observability storage for logs, metrics and traces	₹8 to ₹25 lakh
Platform and SRE people: 2 to 4 engineers, fully loaded	₹50 lakh to ₹1.6 crore

These are planning ranges, not quotes. A well-built modular monolith for the same product might run on a few VMs and one database cluster for a small fraction of this. The extra spend is justified when it buys real team independence; if it does not, it is waste.

08

Security and governance

APIs expose business logic and data directly. The most common API breaches are not clever attacks. They are missing checks, such as letting a logged-in user read someone else's record by changing an ID in the URL. The OWASP API Security Top 10 (2023) is a good working list:

Risk	What it looks like	Control
Broken object-level authorisation	Change /orders/1001 to /orders/1002 and see another customer's order	Check ownership of every object in the service, not only at the gateway
Broken authentication	Weak tokens, keys in URLs, no expiry	OAuth 2.0 and OIDC, short-lived tokens, mTLS for partners
Excessive data exposure	The API returns full records and the app hides fields	Return only the fields the caller needs
Unrestricted resource use	Scraping, brute force, sudden cost spikes	Rate limits and quotas per caller at the gateway
Function-level authorisation	A normal user can call admin endpoints	Separate admin APIs; check roles on every endpoint
Improper inventory	Old versions and test APIs still exposed	An API catalogue, and retire old versions on a schedule

Identity and transport

- Users and apps get tokens from one identity provider using OAuth 2.0 and OIDC; tokens live minutes, not days.
- Machine-to-machine calls use the client credentials flow with narrow scopes, never a shared admin key.
- Partners use mTLS as well as tokens, so a leaked key alone is not enough.
- Inside the platform, encrypt service-to-service traffic with mTLS, through a mesh or the platform's network layer.
- Keep secrets and keys out of code and front-end apps; load them from a vault at run time.

Opening APIs to partners

Partner APIs are a product. Give partners a portal with documentation, a sandbox with realistic test data, self-service keys, published rate limits and a status page. Agree a deprecation policy in the contract. Measure usage per partner, because that is what tells you which versions you can retire and which partners need help.

Governance that teams accept

Publish short API guidelines and enforce them automatically: lint OpenAPI files in CI, run security tests before release, and register every API in the catalogue with an owner. Log every call with caller identity and alert on unusual patterns. For regulated sectors, RBI, SEBI and IRDAI expectations on access control and audit trails, CERT-In 6-hour incident reporting and the DPDP Act all assume you know which APIs touch personal data and who called them.

09

Operating it: day 2

Observability across services

In a distributed system, a single request touches several services, so logs alone are not enough. Give every request a trace ID at the gateway and pass it through every call and event, using OpenTelemetry. Collect three views for every service: the rate of requests, the rate of errors and the duration (the RED method), and for every event consumer, its lag. When something breaks, the trace shows which hop was slow, not just that the page was.

Service levels and error budgets

Set a service level objective per API that matters to users, for example 99.9 percent of requests succeed within 300 milliseconds over 30 days. The gap to 100 percent is the error budget: about 43 minutes a month. When a team has spent it, reliability work comes before new features. This turns reliability arguments into numbers.

Release and change

- Deploy each service independently through its own pipeline; if two must always deploy together, merge them or fix the contract.
- Run consumer-driven contract tests (for example Pact) in CI, so a provider change that breaks a caller fails the build.
- Release behind feature flags or with canary traffic for high-risk changes.
- Track who calls each API version, and retire old versions on the published schedule.
- Review dead-letter queues daily; a growing queue is a silent outage.

Ownership

Every API and every event topic has a named owning team, recorded in the catalogue. The team that builds a service runs it and carries its alerts. A small central platform or integration team owns the gateway, broker, portal and guidelines, and reviews new APIs, but does not build every integration. Central teams that build everything become the bottleneck the architecture was meant to remove.

10

A practical roadmap

Integration improvement works best in steps that each deliver something, without a big-bang rewrite.

Stage	Typical duration	Outcome
1. Map	3 to 4 weeks	Every integration, its owner, protocol and what breaks if it stops. Exposed APIs found and listed.
2. Decide the shape	2 to 4 weeks	Domains and boundaries, where events fit, what stays as it is, API guidelines drafted.
3. Front door	4 to 8 weeks	Gateway, identity, rate limits, catalogue and portal in front of existing APIs. Unused endpoints closed.
4. Make failure safe	4 to 6 weeks	Timeouts, retries, circuit breakers and idempotency on critical paths; tracing end to end.
5. Change gradually	Ongoing, by quarter	Worst point-to-point links replaced first; new features as services or modules; legacy strangled piece by piece.
6. Hand over	4 weeks, overlapping	Guidelines, templates, review practice and runbooks owned by your teams.

11

Ten common mistakes

1. **Splitting before the boundaries are clear.** The result is a distributed monolith that is harder to change than the original.
2. **Sharing a database between services.** No service can change its own data without a coordinated release.
3. **Long chains of synchronous calls.** Availability falls and latency adds up with every hop.
4. **Calls without timeouts.** One slow dependency takes down everything upstream.
5. **Retrying at every layer.** A small fault becomes a flood of traffic at the worst moment.
6. **Payments and orders without idempotency keys.** A retry charges or ships twice.
7. **Breaking changes without a new version.** Partners find out in production.
8. **Authorisation only at the gateway.** Services trust any request that gets past it, and object-level checks are missed.
9. **Business logic in the gateway or the bus.** A new central monolith that one team controls.
10. **No catalogue and no owners.** Forgotten APIs stay open and unmonitored for years.

12

API and integration readiness checklist (36 points)

Use this list to score your current integration estate or a planned programme. Anything you cannot tick with evidence is a gap worth closing.

Architecture and boundaries

- [ ] Domains and boundaries mapped to business capabilities, each with an owner
- [ ] A written reason for every service split, tied to team or scale needs
- [ ] No database shared between services or modules
- [ ] User-facing requests wait on no more than two or three synchronous calls
- [ ] Events used where several systems react to the same change
- [ ] Legacy systems reached only through adapters or the gateway

API design

- [ ] Every API described in OpenAPI or AsyncAPI before build
- [ ] One error format with correlation IDs across all APIs
- [ ] Idempotency keys on payment, order and other write operations that matter
- [ ] Pagination and filtering on every list endpoint
- [ ] Versioning and deprecation policy published, with notice periods
- [ ] Consumer-driven contract tests running in CI

Resilience

- [ ] Timeout on every outbound call, shorter than the caller's timeout
- [ ] Retries limited, with backoff and jitter, at one layer only
- [ ] Circuit breakers on calls to critical dependencies
- [ ] Separate connection pools for each important dependency
- [ ] Outbox pattern used where a database change must produce an event
- [ ] Dead-letter queues monitored and reviewed daily

Security

- [ ] All external traffic through a WAF and the API gateway
- [ ] OAuth 2.0 and OIDC with short-lived tokens; no keys in URLs
- [ ] mTLS for partner and service-to-service traffic
- [ ] Object-level authorisation checked inside every service
- [ ] Rate limits and quotas per caller
- [ ] APIs tested against the OWASP API Top 10 before release
- [ ] Every call logged with caller identity and retained for audit

Governance and partners

- [ ] API catalogue with an owner for every API and event topic
- [ ] Old versions and test endpoints found and retired on schedule
- [ ] Developer portal with documentation, sandbox and self-service keys
- [ ] Usage tracked per partner and per API version
- [ ] API guidelines enforced by linting in CI

Operations

- [ ] Trace ID created at the gateway and passed through every call and event
- [ ] Request rate, error rate and duration dashboards for every service
- [ ] Consumer lag monitored for every event topic
- [ ] Service level objectives and error budgets set for key APIs
- [ ] Runbook for each critical integration, including how to replay events
- [ ] Each service deployable on its own, with rollback practised

13

Glossary

Term	Meaning
Bounded context	A part of the business with its own data, rules and language; a natural service or module boundary.
Modular monolith	One deployable application built from strictly separated modules, each owning its data.
Distributed monolith	Services that must change and deploy together, with the downsides of both styles.
Idempotency	Doing the same operation twice has the same effect as doing it once.
Circuit breaker	A guard that stops calls to a failing dependency for a while, so callers fail fast.
Outbox pattern	Writing an event to the same database transaction as the change, then publishing it.
Strangler pattern	Replacing a legacy system gradually by routing features to new services behind one front.
mTLS	Mutual TLS: both sides of a connection prove their identity with certificates.
Error budget	The amount of failure a service level objective allows over a period.

About Vakratron Systems

Vakratron Systems LLP is a vendor-neutral infrastructure design firm, recognised as a startup by DPIIT. We design data centre, disaster recovery, cloud, GPU and AI platforms for enterprises and government buyers. Every design starts from the workload, compares realistic options with their costs, writes its assumptions down, and is tested before handover.

How we can help with APIs and microservices

- **Integration review:** a map of what talks to what, where it breaks and what to fix first, in two to three weeks.
- **Architecture and API design:** service boundaries, gateway and event design, API guidelines, security controls and a cost view.
- **Partner API launch:** onboarding, keys and mTLS, rate limits, a developer portal and the monitoring to run it.

Talk to us
connect@vakratronsys.com

Online
vakratronsys.com/advisory

This paper gives general guidance based on common enterprise requirements. Figures are planning ranges, not quotations. Apply them to your environment with care, or ask us to help. © 2026 Vakratron Systems LLP.